BITS PILANI DIGITAL

Feature Engineering — Graded Assignment 1

Complete Knowledge Index

Every Topic, Technique, Method & Concept Covered in This Assignment

Abhishek Singh — 2025EM1300088 — BITS Pilani Digital

This document is a complete inventory of every concept, technique, method, tool, and principle encountered in Graded Assignment 1. Use it as a study reference, a knowledge audit, and a guide for what to research further.

Category	Topics Covered	Count
Python Programming	Syntax, data types, control flow, functions, OOP basics	28
Data Analysis	EDA, distributions, correlations, outlier detection	22
Statistics	Skewness, kurtosis, MNAR, Pearson correlation, IQR	18
Feature Engineering	Imputation, encoding, scaling, transformation, construction	31
Machine Learning	RF importance, PCA, MI, pipelines, leakage	19
Python Libraries	pandas, numpy, matplotlib, seaborn, sklearn methods	47
Data Quality	Missing values, outliers, redundancy, multicollinearity	12
Domain Knowledge	HR analytics, attrition drivers, workforce metrics	15
Software Engineering	Pipeline design, reusability, serialisation, version control	8

1. Python Programming Concepts

Every Python syntax element, data structure, and programming pattern used in the notebook.

1.1 Core Syntax & Data Types

1. **Variable Assignment** [Core] — *Storing values with = operator*
2. **String** [Data Type] — *Text data in single or double quotes*
3. **Integer** [Data Type] — *Whole numbers (int64)*
4. **Float** [Data Type] — *Decimal numbers (float64)*
5. **Boolean** [Data Type] — *True/False values from comparisons*
6. **List [ ]** [Data Structure] — *Ordered mutable collection of items*
7. **Dictionary { }** [Data Structure] — *Key-value pairs for lookups and mapping*
8. **Tuple ( )** [Data Structure] — *Immutable ordered collection — used for df.shape*
9. **NaN (Not a Number)** [Special Value] — *Pandas representation of missing data*
10. **None** [Special Value] — *Python's null value — different from NaN*

1.2 Control Flow

11. **for loop** [Control Flow] — *Iterate over a list or range*
    - for col in column_list: — iterating column names
    - for i, col in enumerate(): — getting index AND value together
    - for i in range(len(...)): — numeric index iteration
12. **Nested for loop** [Control Flow] — *Loop inside a loop — used in correlation pair detection*
13. **if / else statement** [Control Flow] — *Conditional execution*
14. **if len(df) > 0 else** [Control Flow] — *Ternary-style condition*

1.3 Functions

15. **def keyword** [Functions] — *Define a reusable function*
16. **Function parameters** [Functions] — *Input variables passed to a function*
17. **return statement** [Functions] — *Send a value back from a function*
18. **lambda function** [Functions] — *Anonymous one-line function — used in FuncFormatter*
    - lambda x, _: f'{x/1000:.0f}K' — formats axis labels in thousands
19. **f-string (f'')** [String Formatting] — *Embed variables inside text strings with { }*
    - f'{value:.3f}' — 3 decimal places
    - f'{value:.1%}' — percentage with 1 decimal
    - f'{value:.0f}K' — zero decimals plus K suffix

1.4 Object-Oriented Basics

20. **Object / Instance** [OOP] — *A specific example of a class — e.g. scaler = StandardScaler()*
21. **Method** [OOP] — *A function belonging to an object — e.g. scaler.fit()*
22. **Attribute** [OOP] — *A property of an object — e.g. pca.explained_variance_ratio_*
23. **Instantiation** [OOP] — *Creating an object from a class with ()*
24. **BaseEstimator / TransformerMixin** [OOP] — *sklearn base classes for custom transformers*

1.5 Operators

25. **Arithmetic operators** [Operators] — *+ - * / for column math*
26. **Comparison operators** [Operators] — *> < >= <= == != for filtering and flagging*
27. **Logical operators** [Operators] — *& (AND), | (OR) for combining conditions*
    - (Promotion_Count == 0) & (Years_at_Company >= 3) — binary flag creation
28. **~ (tilde)** [Operators] — *NOT operator in pandas — inverts boolean Series*
    - ~importance_df['Feature'].isin(protected) — exclude protected features

2. Pandas Library Methods

Every pandas function, method, and accessor used in the notebook.

2.1 DataFrame Creation & Loading

1. **pd.read_csv()** [I/O] — *Load CSV file into a DataFrame*
2. **pd.DataFrame({})** [Creation] — *Create DataFrame from dictionary*
3. **pd.Series()** [Creation] — *Create a Series (single column)*
4. **df.copy()** [Creation] — *Create an independent copy of a DataFrame*

2.2 Inspection & Exploration

5. **df.shape** [Inspection] — *Returns (rows, columns) as a tuple*
6. **df.head(n)** [Inspection] — *First n rows (default 5)*
7. **df.dtypes** [Inspection] — *Data type of each column*
8. **df.describe()** [Inspection] — *Statistical summary of all numeric columns*
9. **df.columns** [Inspection] — *List of all column names*
10. **df.nunique()** [Inspection] — *Number of unique values per column*
11. **df.isnull()** [Inspection] — *True/False DataFrame marking missing values*
12. **df.isnull().sum()** [Inspection] — *Count of missing values per column*
13. **df.duplicated()** [Inspection] — *True/False Series marking duplicate rows*
14. **df.dtypes.value_counts()** [Inspection] — *Count columns by data type*

2.3 Selection & Filtering

15. **df['column']** [Selection] — *Select one column as a Series*
16. **df[['col1','col2']]** [Selection] — *Select multiple columns as DataFrame*
17. **df.loc[condition]** [Selection] — *Filter rows by boolean condition*
18. **df.loc[condition, 'col']** [Selection] — *Filter rows AND select column simultaneously*
19. **df.iloc[i, j]** [Selection] — *Select by integer position (row i, column j)*
20. **df.select_dtypes(include='float64')** [Selection] — *Select columns by data type*
21. **.query('condition')** [Selection] — *SQL-like row filtering*
22. **.isin(list)** [Selection] — *Check if values are in a given list*

2.4 Aggregation & Statistics

23. **.mean()** [Aggregation] — *Column/row average — axis=1 for row-wise*
24. **.median()** [Aggregation] — *Middle value — robust to outliers*
25. **.mode()** [Aggregation] — *Most frequent value(s)*
26. **.std()** [Aggregation] — *Standard deviation*
27. **.sum()** [Aggregation] — *Column total — counts True values when applied to booleans*
28. **.min() / .max()** [Aggregation] — *Minimum and maximum values*
29. **.quantile(0.25)** [Aggregation] — *Percentile — 0.25=Q1, 0.75=Q3*
30. **.skew()** [Aggregation] — *Pearson skewness coefficient per column*
31. **.corr()** [Aggregation] — *Pairwise Pearson correlation matrix*
32. **.corrwith(series)** [Aggregation] — *Correlation of all columns with a specific Series*

33. **.value_counts()** [Aggregation] — *Count occurrences of each unique value*
34. **.value_counts(normalize=True)** [Aggregation] — *Proportions instead of counts*
35. **.groupby().mean()** [Aggregation] — *Mean of a column grouped by another column*

2.5 Transformation & Cleaning

36. **.fillna(value)** [Transformation] — *Replace NaN with specified value*
37. **.dropna()** [Transformation] — *Remove rows or columns containing NaN*
38. **.drop(columns=[])** [Transformation] — *Remove specified columns*
39. **.map(dictionary)** [Transformation] — *Replace values using dictionary lookup*
40. **.astype(type)** [Transformation] — *Convert column data type*
41. **.clip(lower, upper)** [Transformation] — *Cap values at boundaries — IQR capping*
42. **.round(n)** [Transformation] — *Round to n decimal places*
43. **.rename()** [Transformation] — *Rename columns or index*
44. **.reset_index(drop=True)** [Transformation] — *Reset row index to 0,1,2...*
45. **.stack() / .unstack()** [Transformation] — *Reshape wide to long or long to wide*

2.6 Advanced Operations

46. **pd.get_dummies()** [Encoding] — *One-hot encoding of categorical columns*
    - drop_first=True — remove first dummy to avoid dummy variable trap
    - columns=[] — specify which columns to encode
47. **pd.cut()** [Binning] — *Bin continuous values into discrete intervals*
48. **.to_frame()** [Conversion] — *Convert Series to DataFrame*
49. **.to_string()** [Output] — *Convert DataFrame to string for printing*
50. **pd.set_option()** [Configuration] — *Set global pandas display options*

3. NumPy Library Methods

3.1 Mathematical Functions

1. **np.log1p(x)** [Math] — *log(1+x) — safe log transform for zero-containing data*
2. **np.log(x)** [Math] — *Natural logarithm — undefined for x≤0*
3. **np.cumsum()** [Math] — *Cumulative sum — used for PCA variance curve*
4. **np.argmax()** [Math] — *Index of the maximum value in an array*
5. **np.ones_like()** [Array] — *Array of ones with same shape as input*
6. **np.asarray()** [Array] — *Convert input to NumPy array*
7. **np.isnan()** [Boolean] — *True where value is NaN — pipeline verification*
8. **np.inf** [Constants] — *Infinity — used as bins boundary in pd.cut()*
9. **np.random.seed()** [Random] — *Set random seed for reproducibility*

3.2 Array Operations

10. **np.number** [Types] — *Type selector for numeric dtypes in select_dtypes()*
11. **np.triu()** [Array] — *Upper triangle of a matrix — used to mask correlation heatmap*
12. **np.clip()** [Array] — *Numpy version of clipping — used in custom transformer*
13. **np.cumsum()** [Array] — *Running total — converts individual variance to cumulative*

4. Visualization Library Methods

4.1 Matplotlib Core

1. **plt.subplots(rows, cols)** [Layout] — *Create a grid of subplot areas*
2. **axes.flatten()** [Layout] — *Convert 2D subplot grid to 1D array for looping*
3. **fig, axes = plt.subplots()** [Layout] — *Unpack figure container and axes array*
4. **figsize=(width, height)** [Layout] — *Set overall figure size in inches*
5. **plt.figure(figsize=())** [Layout] — *Create a single figure*
6. **plt.show()** [Output] — *Display the chart*
7. **plt.tight_layout()** [Layout] — *Automatically adjust spacing to prevent overlap*
8. **plt.suptitle()** [Labels] — *Overall title across all subplots*
9. **ax.set_title()** [Labels] — *Title for one subplot*
10. **ax.set_xlabel() / set_ylabel()** [Labels] — *Axis labels*
11. **ax.set_xticklabels()** [Labels] — *Custom x-axis tick labels*
12. **ax.tick_params(rotation=)** [Labels] — *Rotate tick labels to prevent overlap*
13. **ax.axhline(y=)** [Reference Lines] — *Draw a horizontal reference line*
14. **ax.axvline(x=)** [Reference Lines] — *Draw a vertical reference line*
15. **ax.annotate()** [Annotations] — *Add text annotation with optional arrow*
16. **ax.hist()** [Charts] — *Histogram of a distribution*
17. **ax.boxplot()** [Charts] — *Box-and-whisker outlier plot*
18. **ax.barh()** [Charts] — *Horizontal bar chart*
19. **ax.fill_between()** [Charts] — *Shade area between a line and baseline*

4.2 Matplotlib Ticker (Axis Formatting)

20. **import matplotlib.ticker as mticker** [Formatting] — *Import ticker submodule for axis formatting*
21. **mticker.FuncFormatter()** [Formatting] — *Custom axis label formatter using a function*
22. **axes[i].xaxis.set_major_formatter()** [Formatting] — *Apply formatter to x-axis*
    - ▸ lambda x, _: f'{x/1000:.0f}K' — converts 80000 to 80K

4.3 Seaborn Charts

23. **sns.set_theme(style=)** [Configuration] — *Set global plot style — whitegrid adds gridlines*
24. **sns.heatmap()** [Charts] — *Colour-coded correlation matrix*
    - ▸ mask=np.triu() — hide upper triangle (duplicate values)
    - ▸ annot=True — show correlation values in cells
    - ▸ cmap='coolwarm' — blue=negative, red=positive
    - ▸ center=0 — white at zero
25. **sns.histplot()** [Charts] — *Histogram with optional KDE curve*
26. **sns.boxplot()** [Charts] — *Box plot from seaborn*
27. **sns.countplot()** [Charts] — *Bar chart counting category frequencies*
28. **sns.barplot()** [Charts] — *Bar chart with confidence intervals*

5. Statistical Concepts

5.1 Descriptive Statistics

1. **Mean** [Central Tendency] — *Sum / count — sensitive to outliers*
2. **Median** [Central Tendency] — *Middle value — robust to outliers*
3. **Mode** [Central Tendency] — *Most frequent value — used for ordinal imputation*
4. **Standard Deviation** [Dispersion] — *Spread around the mean*
5. **Variance** [Dispersion] — *Squared standard deviation*
6. **Minimum / Maximum** [Dispersion] — *Range boundaries*
7. **Percentile / Quantile** [Position] — *Value below which x% of data falls*
8. **Q1 (25th percentile)** [Position] — *First quartile — 25% of data below*
9. **Q3 (75th percentile)** [Position] — *Third quartile — 75% of data below*
10. **IQR (Interquartile Range)** [Dispersion] — *Q3 - Q1 — middle 50% spread, robust to outliers*

5.2 Distribution Shape

11. **Skewness** [Shape] — *Measure of distribution asymmetry*
    - ▸ Positive skew (>0): right tail longer — compensation features
    - ▸ Negative skew (<0): left tail longer — Attendance_Score
    - ▸ |skew| > 1: highly skewed — needs treatment
12. **Right-skewed distribution** [Shape] — *Most values low, few very high — Stock_Option_Value*
13. **Zero-inflated distribution** [Shape] — *Many exact zeros — Stock_Option_Value median = 0*
14. **Normal distribution** [Shape] — *Symmetric bell curve — Training_Hours (skew≈0)*
15. **Long tail** [Shape] — *Extreme values stretching a distribution*

5.3 Correlation & Relationships

16. **Pearson Correlation Coefficient (r)** [Correlation] — *Linear relationship strength — range -1 to +1*
    - ▸ r = 0: no linear relationship
    - ▸ r > 0.85: strong enough to indicate redundancy
    - ▸ r = 0.9989: near-perfect — Annual_Compensation is derived
17. **Multicollinearity** [Correlation] — *When features correlate strongly with each other — confuses linear models*
18. **Dummy variable trap** [Correlation] — *Perfect collinearity between dummy variables — fixed by drop_first*
19. **Mutual Information** [Relationship] — *Non-linear information shared between feature and target*
20. **Correlation matrix** [Correlation] — *All pairwise correlations in one square table*
21. **Upper triangle masking** [Correlation] — *Remove duplicate pairs from correlation heatmap*

5.4 Missing Data Theory

22. **MCAR (Missing Completely At Random)** [Missing Data] — *Missingness unrelated to any variable — simplest case*
23. **MAR (Missing At Random)** [Missing Data] — *Missingness related to other observed variables*
24. **MNAR (Missing Not At Random)** [Missing Data] — *Missingness related to the missing value itself — most complex*
25. **MNAR check** [Missing Data] — *Compare attrition rate in missing vs non-missing rows — test for MNAR*
    - ▸ If rates similar → MCAR → simple imputation justified
    - ▸ If rates very different → MNAR → need missing indicator flag

26. **Missingness threshold** [Missing Data] — *< 10% → impute; 10-30% → impute with caution; > 30% → consider dropping*
27. **Imputation** [Missing Data] — *Filling missing values with estimated values*

5.5 Outlier Theory

28. **Outlier** [Outliers] — *Value that differs significantly from other observations*
29. **Tukey fence** [Outliers] — *Q1-1.5×IQR and Q3+1.5×IQR — standard outlier boundaries*
30. **Genuine vs error outliers** [Outliers] — *Senior executive salary = genuine; impossible Age-Experience = error*
31. **Winsorization / Capping** [Outliers] — *Pull outliers inward to boundary — preserve rank, don't delete*
32. **1st/99th percentile capping** [Outliers] — *More conservative than IQR fences for treatment*

6. Feature Engineering Techniques

6.1 Missing Value Treatment

1. **Median imputation** [Imputation] — *Fill with column median — used for Annual_Bonus (skewed)*
2. **Mean imputation** [Imputation] — *Fill with column mean — used for Training_Hours (symmetric)*
3. **Mode imputation** [Imputation] — *Fill with most frequent value — used for ordinal columns*
4. **Row deletion** [Imputation] — *Removing incomplete rows — rejected here (unnecessary data loss)*
5. **KNN imputation** [Imputation] — *Fill using k nearest neighbours — considered but rejected (low missingness)*
6. **MICE imputation** [Imputation] — *Multivariate imputation by chained equations — rejected (no MNAR evidence)*
7. **Missingness flag** [Imputation] — *Binary column indicating where value was missing — not needed here*

6.2 Outlier Treatment

8. **IQR capping** [Outliers] — *Clip to Q1-1.5×IQR / Q3+1.5×IQR — for moderate skew*
9. **Percentile capping** [Outliers] — *Clip to 1st/99th percentile — for extreme outliers in pipeline*
10. **Log transformation** [Outliers] — *Applies log to compress right tail*
11. **Deletion** [Outliers] — *Removing outlier rows — rejected (loss of valid data)*

6.3 Transformation Techniques

12. **Log transform — log(x)** [Transformation] — *Natural log — undefined for x≤0*
13. **Log1p transform — log(1+x)** [Transformation] — *Safe log for zero values — used for Stock_Option_Value*
14. **Box-Cox transform** [Transformation] — *Optimal power transform — considered but rejected (simpler log1p sufficient)*
15. **Yeo-Johnson transform** [Transformation] — *Handles negative values — not needed here*
16. **Square root transform** [Transformation] — *Milder than log — for moderate skew*
17. **Power transform** [Transformation] — *General-purpose — sklearn's PowerTransformer*

6.4 Encoding Techniques

18. **Ordinal encoding** [Encoding] — *Assign integers preserving rank order — Education, Compensation_Band*
19. **One-hot encoding** [Encoding] — *Binary dummy variables for nominal categories*
20. **drop_first=True** [Encoding] — *Remove one dummy per category to avoid dummy variable trap*
21. **handle_unknown='ignore'** [Encoding] — *Gracefully handle unseen categories at inference time*
22. **Label encoding** [Encoding] — *Simple integer assignment — not used (implies false ordering)*
23. **Target encoding** [Encoding] — *Replace with historical target rate — considered, rejected (leakage risk)*
24. **Binary encoding** [Encoding] — *Two dummies per category — not used here*

6.5 Scaling Techniques

25. **StandardScaler** [Scaling] — *Zero mean, unit variance — (x - mean) / std*
26. **MinMaxScaler** [Scaling] — *Scale to [0,1] range — sensitive to outliers — rejected*
27. **RobustScaler** [Scaling] — *Scale using IQR — robust to outliers*
28. **Normalizer** [Scaling] — *Scale each row to unit norm*
29. **Why scale?** [Scaling] — *Distance-based and gradient algorithms weight large-range features unfairly without scaling*

30. **What NOT to scale** [Scaling] — *Target variable, ordinal integers, binary dummies*

6.6 Feature Construction

31. **Ratio features** [Construction] — *Dividing two columns to create a meaningful proportion*
    - Tenure_Ratio = Years_at_Company / (Total_Work_Experience + 1)
    - Salary_To_Expense_Ratio = Monthly_Salary / (Monthly_Expenses + 1)
    - +1 in denominator prevents division by zero
32. **Difference features** [Construction] — *Subtracting related columns*
    - Experience_Gap = Total_Work_Experience - Years_at_Company
33. **Composite / Aggregate features** [Construction] — *Mean of multiple related columns*
    - Satisfaction_Score = mean of 6 satisfaction sub-scores
34. **Binary flag features** [Construction] — *Threshold-based 0/1 indicators from domain knowledge*
    - High_Overtime_Flag: Overtime > 12 (top quartile from Part A)
    - Career_Stagnation_Flag: Zero promotions AND 3+ years tenure
    - High_Distance_Commuter: Distance > 30km (90th percentile)
35. **Interaction features** [Construction] — *Multiplying two features — not used but common technique*
36. **Attrition lift validation** [Validation] — *Comparing attrition rate in flag=1 vs flag=0 groups*

6.7 Feature Selection

37. **Correlation-based selection** [Selection] — *Remove one of two highly correlated features*
38. **Threshold: |r| > 0.85** [Selection] — *Standard threshold for redundancy*
39. **Random Forest importance** [Selection] — *Tree-based importance — captures non-linear relationships*
40. **Mutual Information** [Selection] — *Information-theoretic importance — captures any relationship*
41. **Threshold: importance < 0.005** [Selection] — *Remove features contributing < 0.5% to prediction*
42. **Dropping identifier columns** [Selection] — *Employee_ID — unique key, no generalizable signal*
43. **Dropping derived columns** [Selection] — *Annual_Compensation — mathematical combination of other features*
44. **Dropping composite duplicates** [Selection] — *Engagement_Index — pre-computed mean of present sub-scores*
45. **Dropping deterministic buckets** [Selection] — *Experience_Band — bucketing of Total_Work_Experience*
46. **Protecting features from dropping** [Selection] — *Binary flags protected despite low RF importance — domain evidence*

7. Scikit-learn Components

7.1 Preprocessing

1. **SimpleImputer** [Preprocessing] — *Fill missing values — strategy: median/mean/most_frequent*
2. **StandardScaler** [Preprocessing] — *Zero mean, unit variance scaling*
3. **OrdinalEncoder** [Preprocessing] — *Encode ordered categories with explicit category order*
4. **OneHotEncoder** [Preprocessing] — *Create binary dummy columns — drop='first' option*
5. **LabelEncoder** [Preprocessing] — *Integer encoding without order — not used here*
6. **PowerTransformer** [Preprocessing] — *Box-Cox/Yeo-Johnson — considered but not used*

7.2 Pipeline & Composition

7. **Pipeline(steps=[])** [Pipeline] — *Chain multiple transformers in fixed sequence*
8. **ColumnTransformer(transformers=[])** [Pipeline] — *Apply different pipelines to different column groups*
9. **fit(X)** [Pipeline] — *Learn parameters from training data*
10. **transform(X)** [Pipeline] — *Apply learned transformation*
11. **fit_transform(X)** [Pipeline] — *Learn and apply in one step*
12. **remainder='drop'** [Pipeline] — *Default ColumnTransformer behaviour — unassigned columns dropped*
13. **named_steps['name']** [Pipeline] — *Access a specific step in a pipeline by name*
14. **get_feature_names_out()** [Pipeline] — *Get output column names from fitted pipeline*

7.3 Model Selection

15. **train_test_split()** [Model Selection] — *Split data into training and test sets*
16. **test_size=0.2** [Model Selection] — *20% held out for testing — 80/20 split*
17. **stratify=y** [Model Selection] — *Preserve class proportions in both splits — critical for imbalanced data*
18. **random_state=42** [Model Selection] — *Set seed for reproducibility*

7.4 Ensemble Methods

19. **RandomForestClassifier** [Ensemble] — *Ensemble of decision trees for classification*
20. **n_estimators=100** [Ensemble] — *Number of trees — more = more stable*
21. **feature_importances_** [Ensemble] — *Attribute storing importance score per feature*
22. **n_jobs=-1** [Ensemble] — *Use all CPU cores for parallel training*

7.5 Dimensionality Reduction

23. **PCA (Principal Component Analysis)** [Dim Reduction] — *Combine correlated features into independent components*
24. **pca.fit(X)** [Dim Reduction] — *Learn principal components from data*
25. **explained_variance_ratio_** [Dim Reduction] — *Array of variance explained by each component*
26. **np.cumsum(explained_variance)** [Dim Reduction] — *Cumulative variance curve — find threshold*
27. **95% variance threshold** [Dim Reduction] — *Standard threshold for number of components to retain*
28. **Scree plot** [Dim Reduction] — *Bar chart of individual component variance — find elbow*
29. **Cumulative variance plot** [Dim Reduction] — *Line chart showing running total — find threshold*

7.6 Feature Selection

30. **mutual_info_classif()** [Feature Selection] — *Mutual information between features and binary target*
31. **discrete_features='auto'** [Feature Selection] — *Auto-detect discrete vs continuous features*

8. Machine Learning Concepts

8.1 Fundamental Concepts

1. **Supervised learning** [Fundamentals] — *Learning from labelled data (Attrition = 0 or 1)*
2. **Binary classification** [Fundamentals] — *Predicting one of two classes — Stayed or Left*
3. **Feature (X)** [Fundamentals] — *Input variable used to make predictions*
4. **Target (y)** [Fundamentals] — *Output variable to predict — Attrition*
5. **Training data** [Fundamentals] — *Data used to fit the model — 4000 rows*
6. **Test data** [Fundamentals] — *Held-out data for evaluation — 1000 rows*
7. **Generalisation** [Fundamentals] — *Model performing well on new unseen data*

8.2 Data Leakage

8. **Data leakage** [Leakage] — *Test data influencing training — invalidates evaluation*
9. **Train-test contamination** [Leakage] — *Fitting scaler on all data before splitting*
10. **Target leakage** [Leakage] — *Using information that wouldn't be available at prediction time*
11. **Split before fit rule** [Leakage] — *Always split first, then fit all transformers on train only*

8.3 Class Imbalance

12. **Class imbalance** [Imbalance] — *Unequal distribution of target classes — 81% stayed / 19% left*
13. **Imbalance ratio** [Imbalance] — *4.25:1 — for every 4 who stayed, 1 left*
14. **Stratified split** [Imbalance] — *Preserve imbalance ratio in both train and test splits*
15. **SMOTE** [Imbalance] — *Synthetic Minority Over-sampling — mentioned as future modelling step*
16. **Class weighting** [Imbalance] — *Penalise misclassification of minority class more*

8.4 Decision Trees & Random Forest

17. **Decision tree** [Trees] — *If-then-else rules learned from data*
18. **Gini impurity** [Trees] — *How mixed the classes are at a node — RF minimises this*
19. **Ensemble method** [Trees] — *Combining multiple models for better results*
20. **Bagging** [Trees] — *Bootstrap aggregating — each tree trained on random sample*
21. **Feature importance** [Trees] — *How much each feature reduces impurity across all trees*
22. **Linear vs non-linear** [Trees] — *RF captures non-linear patterns that correlation misses*

9. Software Engineering & Pipeline Design

9.1 Pipeline Design Principles

1. **Reusability** [Design] — *Fit once, apply to any new data without manual steps*
2. **Consistency** [Design] — *Same transformations applied identically each time*
3. **No data leakage** [Design] — *fit() on train only, transform() on test*
4. **Deployability** [Design] — *Single object that can be saved and loaded in production*
5. **Modularity** [Design] — *Separate sub-pipelines for different column types*

9.2 Serialisation

6. **joblib.dump()** [Serialisation] — *Serialize Python object to disk as .pkl file*
7. **joblib.load()** [Serialisation] — *Reload serialized object from disk*
8. **.pkl file format** [Serialisation] — *Pickle format — binary serialization of Python objects*
9. **Reload verification** [Serialisation] — *transform() on test data after reload to confirm integrity*

9.3 Custom Transformers

10. **BaseEstimator** [Custom] — *sklearn base class providing get_params/set_params*
11. **TransformerMixin** [Custom] — *sklearn mixin providing fit_transform() from fit() and transform()*
12. **fit(self, X, y=None)** [Custom] — *Learn parameters from data — return self*
13. **transform(self, X)** [Custom] — *Apply transformation — return transformed X*
14. **get_feature_names_out()** [Custom] — *Required for column name tracking in ColumnTransformer*

9.4 Google Colab Environment

15. **Google Colab** [Environment] — *Cloud Jupyter notebook environment with free GPU*
16. **drive.mount()** [Environment] — *Mount Google Drive to access persistent files*
17. **Session reset** [Environment] — *Colab disconnects lose all variables — Runtime → Run All recovers*
18. **Runtime → Run All** [Environment] — *Re-execute all cells from top to bottom*
19. **Ctrl + S** [Environment] — *Save notebook to Drive*
20. **Working directory /content/** [Environment] — *Default directory in Colab session*

10. HR Domain Knowledge Applied

10.1 Attrition Drivers

1. **Attendance Score** [Attrition Driver] — *#1 predictor — absenteeism is earliest visible attrition signal*
2. **Financial Stress Index** [Attrition Driver] — *#2 predictor — financial pressure drives exit decisions*
3. **Overtime Hours** [Attrition Driver] — *Burnout indicator — top quartile shows 28.4% vs 15.9% attrition*
4. **Work-Life Balance** [Attrition Driver] — *Low scores associate with higher attrition*
5. **Career Growth Index** [Attrition Driver] — *Stagnation signal — employees without growth tend to leave*
6. **Distance From Home** [Attrition Driver] — *Commute burden — 90th percentile shows 24.7% vs 18.4% attrition*
7. **Compensation Band** [Attrition Driver] — *Low band shows 24.6% vs High band 14.3% attrition*

10.2 Industry Benchmarks Used

8. **10-15% attrition benchmark** [Benchmark] — *Normal range for tech firms — our 19% signals systemic issue*
9. **Top quartile overtime threshold** [Benchmark] — *Overtime > 12 hours = high burnout zone*
10. **90th percentile commute** [Benchmark] — *Distance > 30km = excessive burden*
11. **Stagnation signal** [Benchmark] — *3+ years with zero promotions = classic retention risk*

10.3 Feature Decisions from HR Knowledge

12. **Age vs Experience** [HR Decision] — *Dropped Age — Experience is more HR-relevant and Age carries bias risk*
13. **Components over aggregate** [HR Decision] — *Kept Monthly_Salary/Bonus/Stock separately — each signals differently*
14. **Individual satisfaction scores** [HR Decision] — *Kept 6 sub-scores + built composite — granularity AND summary*
15. **Engagement vs satisfaction sub-scores** [HR Decision] — *Dropped Engagement_Index — sub-scores carry more resolution*

10.4 Military & Leadership Context Applied

16. **Logical consistency checks** [Military Context] — *Military mindset: verify data integrity before trusting it*
17. **Evidence before decision** [Military Context] — *Never apply a technique without data support — MNAR check before imputing*
18. **Alternative approaches documented** [Military Context] — *Consider and reject alternatives — shows analytical depth*
19. **Attrition as operational risk** [Military Context] — *19% annual loss = mission degradation — treat as systemic issue*

11. Key Numbers to Remember

Metric	Value	Significance
Dataset rows	5,000	Sample size for feature engineering
Original columns	39	Starting feature count
Final features	39	After all engineering (coincidentally same)
Attrition rate	19.06%	953 left of 5000 — above industry benchmark
Imbalance ratio	4.25:1	Stayed:Left — requires stratified split
Annual_Compensation r	0.9989	Confirms it is derived — perfect multicollinearity
Engagement_Index r	0.9564	Near-perfect correlation with satisfaction mean
Missing values after imputation	0	All 4 features fully imputed
High_Overtime_Flag lift	+12.5%	28.4% vs 15.9% attrition — strongest binary signal
Features after Part B	48	After dropping 4, one-hot encoding
Features after construction	55	After adding 7 engineered features
Features after correlation drop	51	After removing 4 redundant features
Final shape after RF drop	40	39 features + 1 target Attrition
PCA for 95% variance	24 components	Of 39 features — only 38% reduction, not worth it
Pipeline input	36 columns	Raw df minus Employee_ID, Annual_Compensation, Attrition
Pipeline output	45 columns	After encoding expands 6 categorical to 19 dummies
Train set	4,000 rows	80% stratified
Test set	1,000 rows	20% stratified
Attendance_Score (MI)	0.0407	Highest mutual information with Attrition
Satisfaction_Score (MI)	0.0337	Engineered feature ranks #2 in MI

12. Comparative Decision Points

Every place where a choice was made — showing what was considered and why the final choice was made.

Decision Point	Chosen	Rejected	Reason for Choice
Missing value strategy	Imputation	Row deletion	Loss of 350 valid records unnecessary
Annual_Bonus imputation	Median	Mean	Skew=3.06 — mean inflated by high earners
Training_Hours imputation	Mean	Median	Skew≈0 — symmetric, mean appropriate
Manager_Rating imputation	Mode	Mean	Ordinal scale — 3.4 not a valid rating
Moderate skew treatment	IQR capping	Log transform	Log overkill for 1.0-1.5 skew
Zero-safe log transform	log1p()	log()	log(0) undefined — Stock_Option has zeros
Nominal encoding	One-hot	Ordinal/Label	No rank order in Department, Job_Role etc.
Ordinal encoding	Manual map	One-hot	Education has clear rank — PhD > Masters
Scaler choice	StandardScaler	MinMaxScaler	MinMaxScaler sensitive to residual outliers
Dimensionality reduction	Rejected PCA	Applied PCA	38% reduction insufficient; interpretability lost
Feature importance method	RF + MI both	RF alone	Two methods confirm findings more robustly
Annual_Compensation decision	Drop aggregate	Drop components	Components carry independent signals
Age decision	Dropped	Kept	Correlated 0.94 with experience; age carries bias risk
Engagement_Index decision	Dropped	Kept	r=0.96 with satisfaction sub-scores — pure duplicate
Binary flag validation	Attrition lift	RF importance alone	Lift gives direct business evidence

13. Complete Method & Function Index

Quick reference — every function called in the notebook alphabetically.

Function / Method	Library	Used For
.astype(int)	pandas/numpy	Convert dtype — bool to int for sklearn
.axhline(y=)	matplotlib	Horizontal reference line in charts
.axvline(x=)	matplotlib	Vertical reference line in charts
.clip(lower, upper)	pandas	IQR capping — pull outliers to boundary
.corr()	pandas	Pairwise Pearson correlation matrix
.corrwith(series)	pandas	Correlation of all columns with one Series
.copy()	pandas	Independent DataFrame copy — prevent side effects
.describe()	pandas	Statistical summary (count, mean, std, min, quartiles, max)
.drop(columns=[])	pandas	Remove specified columns
.dropna()	pandas	Remove rows with missing values
.dtypes	pandas	Data type of each column
.duplicated()	pandas	Boolean Series marking duplicate rows
.fillna(value)	pandas	Replace NaN with given value
.flatten()	numpy	Convert 2D subplot grid to 1D array
.get_dummies()	pandas	One-hot encode categorical columns
.groupby().mean()	pandas	Mean by group — attrition rate by flag value
.head(n)	pandas	First n rows of DataFrame
.isin(list)	pandas	Boolean check if values are in list
.isnull()	pandas	Boolean DataFrame marking missing values
.isnull().sum()	pandas	Count missing values per column
.iloc[i,j]	pandas	Integer-position row/column selection
.loc[condition]	pandas	Boolean row selection
joblib.dump()	joblib	Save Python object to disk
joblib.load()	joblib	Load saved Python object from disk
.map(dict)	pandas	Replace values using dictionary
.mean(axis=1)	pandas	Row-wise mean across columns
.median()	pandas	Median value
.mode()[0]	pandas	Most frequent value
.mode()[0]	pandas	Get first mode value from Series
mutual_info_classif()	sklearn	Mutual information feature importance
np.argmax()	numpy	Index of maximum value
np.clip()	numpy	Clip array values to bounds
np.cumsum()	numpy	Cumulative sum
np.inf	numpy	Infinity — used in pd.cut() bins
np.isnan()	numpy	Boolean where values are NaN
np.log1p()	numpy	Safe log transform for zero values
np.ones_like()	numpy	Array of ones matching input shape
np.triu()	numpy	Upper triangle of a matrix
OrdinalEncoder(categories=[])	sklearn	Ordinal encoding with explicit rank
PCA()	sklearn	Principal component analysis
pd.cut()	pandas	Bin continuous values into categories
pd.DataFrame({})	pandas	Create DataFrame from dictionary
pd.get_dummies()	pandas	One-hot encoding

Function / Method	Library	Used For
pd.read_csv()	pandas	Load CSV to DataFrame
pd.Series()	pandas	Create a labeled series
pd.set_option()	pandas	Set pandas display options
Pipeline(steps=[])	sklearn	Chain transformers in sequence
plt.figure()	matplotlib	Create a single figure
plt.show()	matplotlib	Display chart
plt.subplots()	matplotlib	Create grid of subplot axes
plt.tight_layout()	matplotlib	Auto-fix subplot spacing
quantile(0.25/0.75)	pandas	Q1 and Q3 for IQR calculation
RandomForestClassifier()	sklearn	Tree ensemble for importance
.reset_index(drop=True)	pandas	Reset row index
.round(n)	pandas	Round to n decimal places
select_dtypes(include=)	pandas	Select columns by dtype
.shape	pandas	Return (rows, cols) tuple
SimpleImputer(strategy=)	sklearn	Fill missing values in pipeline
.skew()	pandas	Skewness per column
sns.barplot()	seaborn	Bar chart
sns.boxplot()	seaborn	Boxplot
sns.countplot()	seaborn	Count bar chart
sns.heatmap()	seaborn	Colour-coded correlation matrix
sns.histplot()	seaborn	Histogram with KDE
sns.set_theme()	seaborn	Global chart styling
StandardScaler()	sklearn	Zero mean unit variance scaling
.sum()	pandas	Column sums
train_test_split()	sklearn	Split into train and test sets
ColumnTransformer()	sklearn	Route columns to different pipelines
.value_counts()	pandas	Count unique values
.value_counts(normalize=True)	pandas	Proportion of unique values
.where(~mask).stack()	pandas	Extract correlation pairs from matrix

This Knowledge Index was prepared by Claude (Anthropic) for Abhishek Singh — 2025EM1300088 — BITS Pilani Digital — Feature Engineering Graded Assignment 1 — July 2026.